Marin in depth: corpus pipelines, mixture proxies, and Delphi

> **Note**
>
> For direct replication, use the Datakit code manual and optimizer-search walkthrough. They distinguish historical Harrier recipes, the current Luxical DAG, reference Vizier tuning, May/August multiplier grids and later hero configurations.

> **Note**
>
> For the latest conversation evidence and **96-GPU × 30-day** allocation, see the industry audit, data-engineering companion, trainer playbook, checkpoint evaluation, and current numerical plan. Earlier 128×40 examples remain historical alternatives.

Marin exposes data pipelines, experiment definitions, scaling heuristics and failed runs. Its data-mixture effort and its dense Delphi suite are related examples, but they are different experiments. Keep their model families, token horizons and corpora separate.

Map the efforts before copying a recipe

Effort	Scientific question	Reported setup
Datakit data pipeline	How to make many upstream datasets usable together	Normalize, globally deduplicate, decontaminate, annotate buckets
Mixture swarm / later MoE hero development	Which bucket weights improve target-like exposures?	40 topics × 5 quality bands; 872 proxy observations in the report
Dense Delphi	Does a prescribed recipe extrapolate across compute?	Qwen-3 family; Nemotron-CC + StarCoderData + ProofPile 2; IsoFLOP fits and larger holdouts
Later efficiency/MoE work	Do combined architecture and systems changes help in practice?	Separate model/optimizer/hardware configurations

Sources: Marin data pipeline report, Delphi report, and efficiency follow-up. The downloaded Markdown mirrors preserve these reports, but some interactive figure/calculator data require the original site or linked artifacts.

Build a reproducible ingestion graph

The data report records upstream provenance and version information, then normalizes into Parquet containing text, a stable text hash, `source_id`, and source-specific metadata. Empty records and formatting pathologies are handled before corpus-wide processing. The pipeline generally references upstream data rather than republishing every source as a single new monolithic corpus.

This has a practical implication for “start with Marin data”: specify the **catalogue, pipeline commit, source revisions and mixture**, not just the project name. A pipeline can be reproducible while an upstream source later becomes unavailable. Verify that the required inputs can actually be materialized before committing to a training plan. Data pipeline report, “Download and normalize”.

Make repetition and contamination controlled quantities

Marin applies exact deduplication using normalized text hashes and fuzzy candidate discovery using MinHash/LSH over word n-grams. It reports removal of 2.33B documents and 2.13T tokens in the described pool. A major overlap is between processed Common Crawl and synthetic rewrites derived from that crawl.

The report also documents false-positive risks for structured data and genomic sequences: small edits can be semantically important despite high text similarity. This is a reason to inspect clusters and apply domain-aware rules, not to apply one text threshold blindly to every modality.

For decontamination, the described implementation compares paragraph-level 13-word n-gram overlap against evaluation material and flags documents when at least half of eligible occurrences in a paragraph match. The authors state that this favors precision and misses some paraphrases. This is a reported heuristic, not a guarantee of a clean benchmark. Data pipeline report, deduplication/decontamination sections.

Bucket the data and model exposure

The reported mixture procedure crosses 40 topics with five quality bands to obtain 200 buckets. Embeddings support lightweight topic and quality labeling; relatively balanced topic sizes help keep sampling weights experimentally meaningful. Available tokens are counted separately from intended training fractions.

For bucket j, average exposure is $e_j = Dw_j/U_j$. The later hero ladder described in the report uses about 791 tokens per active parameter to correspond to an 18T-token run at 22.76B active parameters. These numbers belong to the later MoE effort, not the dense Delphi recipe. Data pipeline report, mixture section and footnotes.

Proxy training tokens and each bucket's available pool are both downscaled. If $D_p = sD_t$ and $U_{jp} = sU_{jt}$, the same mixture weights preserve e_j. Without pool downscaling, a tiny proxy might favor a scarce source because it never reaches the target's repetition regime.

The report fits a mixture-response model from 872 proxy observations and describes a larger-model comparison protocol; its transfer caveat does not establish a measured larger-model win. A later, separate completed comparison in issue #9126 reports measured final gains, with recipe and throughput qualifications explained in the artifact-lineage guide. A per-bucket correlation is not an isolated causal effect because the weights sum to one. The chosen substitution and the multivariate response model matter.

Delphi: tune a recipe before fitting a law

Delphi separates three objects:

1. A **heuristic/recipe** maps model size and token budget to a complete training configuration.
2. An **IsoFLOP experiment** compares size-versus-duration choices under that recipe.
3. A **scaling law** fits the measured outcomes and predicts larger budgets.

The dense suite holds fixed its architecture family, tokenizer, data mixture, WSD schedule, precision convention and parallelism. It fits seven compute budgets from 3×10^{18} to 3×10^{20} FLOPs, then evaluates larger runs. These budget boundaries are historical examples, not required budgets for our project. Delphi, "Scaling recipe procedure".

Why the first recipe failed

The first recipe gave smooth small-scale slice minima, but its 10^{22}-FLOP holdout missed by 2.5% and its 10^{23} run diverged. The report traces a concern to increasing batch size to hold step counts roughly fixed, paired with LR growth. It is not sufficient to say the fit was good: the large-horizon behavior of the update was wrong for that recipe. Delphi, attempt 1.

What changed in the second recipe

The revised recipe uses AdamH, retuned reference settings, and an explicit token-horizon LR correction proportional to $(T_0/T)^{0.3}$. Its width-transfer sanity check uses four-layer models with widths 128–1,024; this tests width behavior at fixed depth rather than proving depth transfer. The second attempt forecasts the largest reported holdout to within roughly 0.2% under that suite's evaluation. That result is evidence for that tested recipe and range, not a universal accuracy tolerance. Delphi, attempt 2 and forecast sections.

The full hyperparameter function is more than one LR exponent. It includes scalar versus projection parameter groups, optimizer moments, epsilon, initialization, clipping, architecture ratios and schedule. Consult the source code for the entire transformation; a partly blank interactive calculator in a Markdown snapshot is not an executable specification.

How the training machinery supports these experiments

The checked-out recipe guide describes this sequence:

1. Establish a promising reference result, often around 130M parameters and 2.5B tokens in the example recipe.
2. Tune reference hyperparameters with a sweep.
3. Implement a `ScalingHeuristic` that emits model, token and optimizer configurations for candidate compute budgets.
4. Run and analyze IsoFLOP candidates.
5. Generate larger predicted configurations and train the ladder.
6. Promote a selected configuration to a recurring small canary.

Relevant files in the local checkout include `experiments/references/reference_hyperparameter_sweep.py`, `completed_adamh.py`, `reference_scaling_suite.py`, and `lib/marin/src/marin/scaling_laws/isoflop_analysis.py`. The checkout snapshot inspected for this guide is commit `1d274b52c636b6f2a9177b1d4c14b2f74795a8a6`; it is not asserted to be the exact historical Delphi revision. Pinned recipe guide.

Its experiment graph also preserves dependencies between downloads, tokenization, training and analysis. The useful design principle is to make each run's inputs and parents explicit. Adopting that principle does not require using the same TPU stack.

What we should adopt and revalidate

Adopt: explicit source lineage; global overlap accounting; fixed validation data; target-matched exposure in proxies; a complete versioned heuristic; size/token variation; real holdouts; and production-stack canaries.

Revalidate: bucket taxonomy, quality model, epoch caps, optimizer, numerical precision, shape progression, token horizon and throughput. Nemotron-CC alone is not the same training distribution as the dense Delphi mixture. A recipe that transfers across width can still fail after changing depth or duration.

Local sources

- Data pipeline snapshot.
- Delphi snapshot.
- Efficiency snapshot.
- Marin code reading map; code archives and commit information are stored with the reports, while `tmp/marin/` is the ignored working checkout.

Continue to the beginning-to-end design, compare OLMo's experiments, or return to the blog reading guide.